

Lustrean

Lean + Lustre

Arthur ADJEDJ

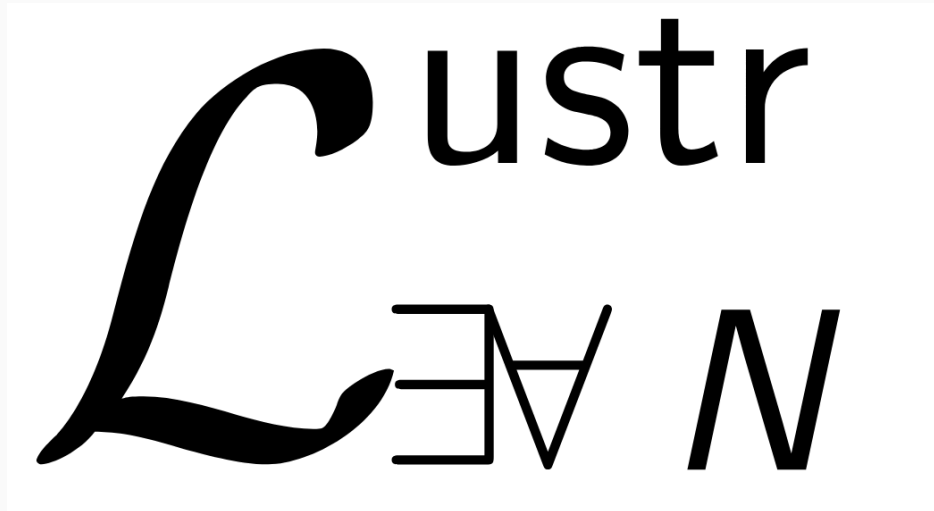
Fernando LEAL SANCHEZ

Léo LEESCO

2026-01-09



 `arthur-adjedj/lustrean`



Lustrean is an abstract interpreter of a precise semantics of a core subset of Lustre in Lean, implemented last year by Jean Caspar and Adrien Mathieu.

- It embeds a subset of Lustre as a DSL in Lean.
- It gives a precise semantics of that language, even to non-statically schedulable programs.
- It interprets them in an abstract domain to check for the absence of runtime errors.

Refactors, debugging, and the hopes for a concrete interpreter

The original Lustrean project was great a great toy project, but needed lots of reworks to make it scalable and maintainable:

- Did not respect the usual code conventions
- Lots of hardcoded features, no good debugging infrastructure
- Could not rely on Mathlib, or any of the lean recent features

The original Lustrean project was great a great toy project, but needed lots of reworks to make it scalable and maintainable:

- Did not respect the usual code conventions
 - Refactored the entire project to respect conventions, and added lots of docs
- Lots of hardcoded features, no good debugging infrastructure
- Could not rely on Mathlib, or any of the lean recent features

The original Lustrean project was great a great toy project, but needed lots of reworks to make it scalable and maintainable:

- Did not respect the usual code conventions
 - Refactored the entire project to respect conventions, and added lots of docs
- Lots of hardcoded features, no good debugging infrastructure
 - Added great debug traces that allowed us to fix many bugs in the initial implementation
 - Added a test suite and a CI to check for breakages
- Could not rely on Mathlib, or any of the lean recent features

The original Lustrean project was great a great toy project, but needed lots of reworks to make it scalable and maintainable:

- Did not respect the usual code conventions
 - Refactored the entire project to respect conventions, and added lots of docs
- Lots of hardcoded features, no good debugging infrastructure
 - Added great debug traces that allowed us to fix many bugs in the initial implementation
 - Added a test suite and a CI to check for breakages
- Could not rely on Mathlib, or any of the lean recent features
 - Made the project compatible with the most recent release (v4.27.0-rc1)
 - Refactored the formalisation of Domain Theory to rely on Mathlib (See Fernando's part)

- ☐  **feat: DOT widget** ✓
#12 opened 2 days ago by arthur-adjedj
- ☐  **feat: change defs to mathlib** ✓
#11 by ayhon was merged 2 days ago
- ☐  **[Need review] reject ill-formed programs** ✗
#10 opened last week by leo-leesco
- ☐  **feat: select custom domains** ✓
#9 by ayhon was merged last week
- ☐  **feat: add 2025 presentation** ●
#8 opened 3 weeks ago by ayhon
- ☐  **fix: do not reinitialize bvars to nil after unrolling the main loop** ✓
#7 by arthur-adjedj was merged 5 days ago

Lean is extensible, let's make use of that.

The elaborator already has a great API for traces, we thus extend it to print our own traces.

```
initialize
  registerTraceClass `Lustrean.Elab (inherited := true)

withTraceNode `Lustrean.Elab.Reify (msg := fun e => return m!"{exceptEmoji e}
elabExpr\n{s}\n⇒\n{e.toOption.map toString}") do ...
```

arthur.typ

Sign.lean

Example.lean M ×

Elaboration.lean

Corr ...

Example.lean

You, 5 seconds ago | 3 authors (ayhon and others)

1

import Lustrean

2

3

set_option trace.Lustrean.Elab.Reify true

4

lustre (trace)

5

node inc(x) = o where

6

o = x + 1

7

Lean InfoView ×

▼ Example.lean:4:6

▼ Messages ⓘ 1

▼ Example.lean:4:0

[Lustrean.Elab.Reify] ✓ elabNode

node inc(x) = o where o = x + [1, 1]

⇒

node inc(x) = o

where

o = x + 1 ▼

[] ✓ elabExpr

x + [1, 1]

⇒

some (x + 1) ▼

[] ✓ elabExpr

x

⇒

some (x)

[] ✓ elabExpr

[1, 1]

⇒

some (1)

► All Messages

Graphviz widget

ation.leanCompile.lean MGraphviz.lean M

Example.lean
You, 25 minutes ago | 3 authors (ayhon and others)
1 import Lustrean
2
3 set_option trace.Lustrean.Elab.DOT true
4 lustre
5 node inc(x) = o where ayhon, last week • fix: revert Example.lean
6 | o = x + 1
7

Lean InfoView ×arthur.typ (Preview)

▼ Example.lean:5:23
▼ HTML Display

```
graph TD; 0((0)) -- "step := 0" --> 2(((2))); 2 -- "x2 := nil" --> 3((3)); 3 -- "x3 := x2" --> 4((4)); 4 -- "x1 := [-∞, ∞]" --> 5((5)); 5 -- "step := (step + 1)" --> 3; 5 --> 5;
```

The hope for a concrete interpreter

Lean has dependent types

Objective: use those to make a correct-by-construction concrete interpreter

How: using CoStreams and heterogeneous lists:

```
def HList : List (Type u) → Type u
| [] => PUnit
| α::tl => α × HList tl

structure CoStream (σ α : Type _) where
  init : σ
  step : σ → α × σ
```

Concrete interpretation example

```
def x {σ} (i : CoStream σ ValType) : CoStream (HList [ValType,
σ]) ValType :=
  let pre_i := i.pre none
  let one := CoStream.const (some 1)
  let pre_i_add_1 := pre_i.map2 (c2 := one) (· + ·)
  pre_i_add_1
```

```
node f(i) = o where
  x = pre i + 1
  o = x+i
```

```
def o {σ} (i : CoStream σ ValType) (x : CoStream (HList
[ValType, σ]) ValType) : CoStream (HList [HList [ValType, σ],
σ]) ValType :=
  let x_add_1 := x.map2 (c2 := i) (· + ·)
  x_add_1
```

```
def f {σ} (i : CoStream σ ValType) :=
  let x := x i
  let o := o i x
  o
```

Good ideas, bad execution.

- The codebase does not already have a scheduler, having a concrete interpreter would mean suddenly refusing more programs

Good ideas, bad execution.

- The codebase does not already have a scheduler, having a concrete interpreter would mean suddenly refusing more programs
- Turns out dependent types are hard..

Good ideas, bad execution.

- The codebase does not already have a scheduler, having a concrete interpreter would mean suddenly refusing more programs
- Turns out dependent types are hard..
- I spent too much time on refactors instead of this 🧡

Good ideas, bad execution.

- The codebase does not already have a scheduler, having a concrete interpreter would mean suddenly refusing more programs
- Turns out dependent types are hard..
- I spent too much time on refactors instead of this 😊

In the end:

- Basic API for `CoStreams` and `HList`, allowing for efficient implementation of lustre programs *by hand*
- Hopes and dreams I will find time to complete the implementation someday

Extending Lustrean

New primitives

Syntax.lean

```
syntax " pre " lustre_expr : lustre_expr
```

Reify.lean

```
inductive MonOp where
```

```
| «pre»
```

```
deriving Repr, Inhabited
```

```
namespace MonOp
```

```
protected def toString : MonOp → String
```

```
| «pre» => "pre"
```

Normalize.lean

```
partial def elabExprAux {n m : Nat} (nod : NodeN n m) : Indicise.Expr n m →
  NormalizeM (AuxExpr n m)
| .mon_op .pre ⟨e, _⟩ => do
  let ⟨m', _, e, nod⟩ ← elabExprAux nod e
  let ⟨m', _, x, nod⟩ ← addVarIfNotBvar default e nod
  return {
    m' := m'
    e := .simple <| .var (.old_bound_var x)
    nod := nod
  }
```

Syntax.lean

```
syntax:35 lustre_expr:36 " fby " lustre_expr:35 : lustre_expr  
syntax:35 lustre_expr:36 " -> " lustre_expr:35 : lustre_expr
```

Reify.lean

```
namespace Reify  
inductive BinOp where  
  | fby  
  | arr  
  deriving Repr, Inhabited  
  
namespace BinOp  
protected def toString : BinOp → String  
  | .fby => "fby"  
  | .arr => "->"
```

Normalize.lean

```

partial def elabExprAux {n m : Nat} (nod : NodeN n m) : Indicise.Expr n m →
  NormalizeM (AuxExpr n m)
  | .bin_op .fby ⟨e₁, _⟩ ⟨e₂, _⟩ => do
    let ⟨_, m_leq_m₁, e₁, nod⟩ ← elabSimpleExprAux nod e₁
    let ⟨m₂, _, e₂, nod⟩ ← elabExprAux nod (e₂.upcast m_leq_m₁)
    let ⟨m', _, x, nod⟩ ← addVarIfNotBvar default e₂ nod
    let cond := .cmp_op .eq (.var .step) (.interval 0 0)
    return {
      m' := m'
      e := .ite cond (e₁.upcast <| by omega) (.var <| .old_bound_var x)
      nod := nod
    }

```

```
| .bin_op .arr ⟨e1, _⟩ ⟨e2, _⟩ => do
  let ⟨_, m_leq_m1, e1, nod⟩ ← elabSimpleExprAux nod e1
  let ⟨m2, _, e2, nod⟩ ← elabSimpleExprAux nod (e2.upcast m_leq_m1)
  let cond := .cmp_op .eq (.var .step) (.interval 0 0)
  return {
    m' := m2
    e := .ite cond (e1.upcast <| by omega) (e2.upcast <| by omega)
    nod := nod
  }
```


Rejecting ill-formed programs

Examples

`node f(x) = x where`

`node f(x,x) = y where`

`node f(x) = y,y where`

→ here we can redefine $z = y$ in the body

`node f() where`

`x = 0`

`x = 1`

`node f() = o, o where`

`o = o`

`node g(x,x) where`

`node h(x) = x where`

Rejecting ill-formed programs

```
node i(x,y) = 0 where  
  0 = x  
  0 = y
```

Rejecting ill-formed programs

Implementation

```
def elabNode (s : TSyntax `lustre_node) : CoreM (&Node) :=
  match s with
  | `(lustre_node| node $name($inputs:ident,*) $[= $output_vars,*]? $[guard
    $guards*]?
      where $decls* $[assert $asserts*]?) =>
    let name := {name.getId, name}
    let input_vars := inputs.getElems.map fun x => {x.getId, x}

    /- we reject programs with input having repetitions -/
    if !input_vars.allDiff then throwIllFormedSyntax
```

Rejecting ill-formed programs

```
let bound_vars ← decls.mapM fun
  | `(lustre_node_decl| $vars:ident,* = $expr:lustre_expr) => do pure {
    names := vars.getElems.map fun var => ⟨var.getId, var⟩
    value := ← elabExpr expr
  }
  | ref => withRef ref throwUnsupportedSyntax

/- we reject programs with multiple redefinitions of the same variable -/
if !(bound_vars.map (BoundVars.names)).allDiff then throwIllFormedSyntax
```

Rejecting ill-formed programs

```
let output_vars : Array &Name := output_vars.map (·.getElems.map (fun var
=> {var.getId, var})) |>.getD default

/- we reject programs with output having repetitions -/
if !output_vars.allDiff then throwIllFormedSyntax

/- we reject programs with input n output  $\neq \emptyset$  -/
if !(intersect (input_vars.map (Variable.name) ) output_vars).isEmpty then
throwIllFormedSyntax

let guards ← guards.getD #[] |>.mapM elabBoolExpr
let asserts ← asserts.getD #[] |>.mapM elabBoolExpr
return {name, input_vars, bound_vars, output_vars, guards, asserts}, s)
| _ =>
throwUnsupportedSyntax
```

Type checking

```
inductive Ty where
  | int : Ty
  | bool : Ty
deriving BEq
```

- integer operations $\rightarrow$ type checks for `int` on operands
- boolean operations $\rightarrow$ type checks for `bool` on operands
- comparisons $\rightarrow$ type checks for same types on operands, constrains operands to the same type in case not typed yet

Design choices

The type-checker does not care about the actual implementation, and focuses only on the carried type (anyway, it is thrown away at runtime).

What's left ?

Finish the type-checker

Language extensions

when, merge

Type checking clocks

Currently hard to do because we currently rely on a program-wide clock, which we cannot type without doing a complete overhaul of the system.

New in the abstract interpreter

The quest towards the partition
domain

Recap of what we have

Recap of what we have

- Interval value domain

Recap of what we have

- Interval value domain
- Undefined value domain combinator

Recap of what we have

- Interval value domain
- Undefined value domain combinator
- NonRelationa domain combinator

Recap of what we have

- Interval value domain
- Undefined value domain combinator
- NonRelationa domain combinator

We want more

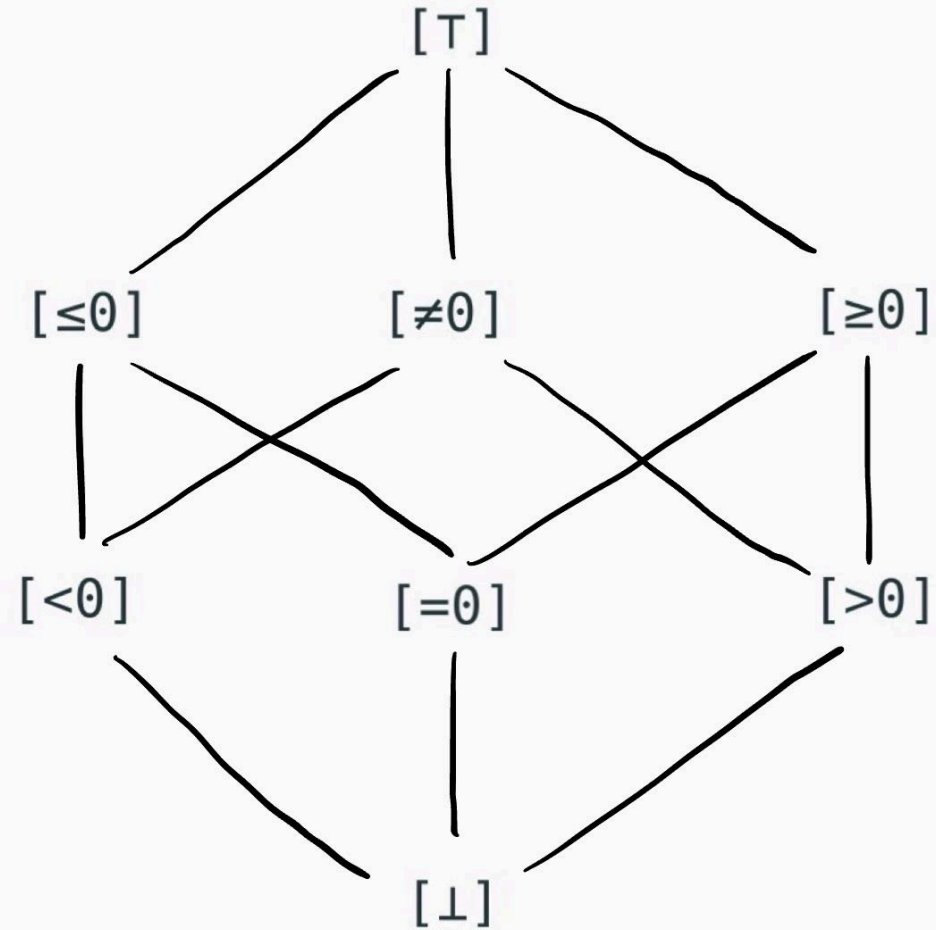
Two ways to increase the amount of domains we have

- New basic domains
- New domain combinators

Two ways to increase the amount of domains we have

- New basic domains ← Sign value domain
- New domain combinators ← Partition domains

Sign domain - Definition



Sign domain - Definition

Sign		Interval
$[\perp]$	$\mapsto$	$\emptyset$
$[\top]$	$\mapsto$	$(-\infty, +\infty)$
$[=0]$	$\mapsto$	$[0, 0]$
$[>0]$	$\mapsto$	$[0, +\infty)$
$[<0]$	$\mapsto$	$(-\infty, 0)$
$[\geq 0]$	$\mapsto$	$[0, +\infty)$
$[\leq 0]$	$\mapsto$	$(-\infty, 0]$
$[\neq 0]$	$\mapsto$	????

Sign domain - Definition

<u>Sign</u>		<u>Interval</u>
$[\perp]$	$\mapsto$	$\emptyset$
$[\top]$	$\mapsto$	$(-\infty, +\infty)$
$[=0]$	$\mapsto$	$[0, 0]$
$[>0]$	$\mapsto$	$[0, +\infty)$
$[<0]$	$\mapsto$	$(-\infty, 0)$
$[\geq 0]$	$\mapsto$	$[0, +\infty)$
$[\leq 0]$	$\mapsto$	$(-\infty, 0]$
$[\neq 0]$	$\mapsto$	$[<0] \sqcup [>0]$

Sign domain - Demo!

Consider the following:

The interval domain is not able to prove this 😞

```
lustre
node f(x) = y
where    y = if x ≥ 0 then x + 1 else x - 1
assert   y ≠ 0
```

Sign domain - Demo!

Consider the following:

However, the sign domain does! 😊

```
lustre (domain := Sign)
  node f(x) = y
  where    y = if x ≥ 0 then x + 1 else x - 1
  assert   y ≠ 0
```

We registered new domains by implementing the `ValueDomain` typeclass

Sign domain - Implementation

We registered new domains by implementing the ValueDomain typeclass

```
class ValueDomain ( $\alpha$  : Type)[BEq  $\alpha$ ]  
extends Add  $\alpha$ , Neg  $\alpha$ , Mul  $\alpha$ , Sub  $\alpha$ , Div  $\alpha$ , BoundedLattice  $\alpha$ ,  
        ToString  $\alpha$ , WidenLawful  $\alpha$ , NarrowLawful  $\alpha$   
where  
  -- interval [a, b]  
  rand : Option Int  $\rightarrow$  Option Int  $\rightarrow$   $\alpha$   
  nil :  $\alpha$   
  dec_bot: DecidablePred ( $\cdot$  = bot) := by  
    exact fun x => (inferInstance: Decidable (x =  $\perp$ ))  
  -- compare op x y = (x', y') such that  
  -- x' = { v  $\in$  x |  $\exists$  v'  $\in$  y, v op v' }  
  -- y' = { v'  $\in$  y |  $\exists$  v  $\in$  x, v op v' }  
  compare : CompareOp  $\rightarrow$   $\alpha \rightarrow \alpha \rightarrow \alpha \times \alpha$ 
```

Sign domain - Implementation

We registered new domains by implementing the `ValueDomain` typeclass which itself makes use of the `BoundedLattice` typeclass.

```
class BoundedLattice ( $\alpha$  : Type) where
  // ...
  join_commutative :  $\forall$  (x y :  $\alpha$ ), join x y = join y x
  join_associative :  $\forall$  (x y z :  $\alpha$ ), join (join x y) z = join x (join y z)
  join_absorption  :  $\forall$  (x y :  $\alpha$ ), join x (meet x y) = x
  join_bot         :  $\forall$  (x :  $\alpha$ ), join x bot = x
  join_top         :  $\forall$  (x :  $\alpha$ ), join x top = top
  meet_commutative :  $\forall$  (x y :  $\alpha$ ), meet x y = meet y x
  meet_associative :  $\forall$  (x y z :  $\alpha$ ), meet (meet x y) z = meet x (meet y z)
  meet_absorption  :  $\forall$  (x y :  $\alpha$ ), meet x (join x y) = x
  meet_top        :  $\forall$  (x :  $\alpha$ ), meet x top = x
  meet_bot        :  $\forall$  (x :  $\alpha$ ), meet x bot = bot
  non_trivial      : top  $\neq$  bot
  := by decide
```

A **Galois connection** between two partially ordered sets A and C (referred to respectively as *abstract domain* and *concrete domain*) is a pair of functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ such that $\forall (a : A)(c : C)$

$$\alpha c \leq a \iff c \leq \gamma a$$

A function g is an **abstraction** of f if the *concretization of its outputs* contains the result over the concretized inputs. (In short, sound)

$$\forall (a : A), f(\gamma a) \leq \gamma(g a)$$

A function g is an **binary abstraction** of f if the *concretization of its outputs* contains the result over the concretized inputs. (In short, sound)

$$\forall (aa' : A), f(\gamma a)(\gamma a') \leq \gamma(g a a')$$

A function g is an **binary perfect abstraction** of f if the *concretization of its outputs* corresponds to the result over the concretized inputs. (In short, sound and complete)

$$\forall (aa' : A), f(\gamma a)(\gamma a') = \gamma(g a a')$$

Sign domain - Implementation

```
-- Abstraction over sets of integers. The only
   information retained is the sign of the elements
   of the set.  -/
structure Sign where mk ::
  hasPos: Bool := false
  hasZero: Bool := false
  hasNeg: Bool := false
  deriving DecidableEq, Repr, Inhabited
```

```
def GaloisConnection
```

[source](#)

```
  { $\alpha$  : Type  $u$ } { $\beta$  : Type  $v$ } [ $Preorder$   $\alpha$ ] [ $Preorder$   $\beta$ ]  
  ( $\downarrow$  :  $\alpha \rightarrow \beta$ ) ( $\uparrow$  :  $\beta \rightarrow \alpha$ ) :
```

Prop

A Galois connection is a pair of functions $\downarrow$ and $\uparrow$ satisfying $\downarrow a \leq b \leftrightarrow a \leq \uparrow b$. They are special cases of adjoint functors in category theory, but do not depend on the category theory library in mathlib.

▼ Equations

- `GaloisConnection` $\downarrow \uparrow = \forall (a : \alpha) (b : \beta), \downarrow a \leq b \leftrightarrow a \leq \uparrow b$

Sign domain - Implementation

```
@[grind =]
noncomputable
def abstract(X: Set Int): Sign := {
  hasZero := 0 ∈ X
  hasPos  := ∃ z ∈ X, z > 0
  hasNeg  := ∃ z ∈ X, z < 0
}
```

```
@[grind =]
def concrete(a: Sign): Set Int :=
  setOf λ z ↦ match compare z 0 with
  | .lt => a.hasNeg
  | .eq => a.hasZero
  | .gt => a.hasPos
```

Sign domain - Implementation

```
def gc: GaloisConnection Sign.abstract Sign.concrete := by
  rintro X ⟨p,z,n⟩
  constructor <;> grind [LE.le]
```

Sign domain - Implementation

```
def gc: GaloisConnection Sign.abstract Sign.concrete := by
  rintro X ⟨p,z,n⟩
  constructor <;> grind [LE.le]
```



Sign domain - Implementation

noncomputable

```
instance ge: GaloisEmbedding abstract concrete := gc.toGaloisInsertion <| by
  rintro ⟨hasPos, z, hasNeg⟩
  simp [Sign.abstract, Sign.concrete, LE.le, Sign.incl]
  apply And.intro
  · if h: hasPos then apply Or.inr; exists 1 else grind
  · if h: hasNeg then apply Or.inr; exists -1 else grind
```

Sign domain - Implementation

```
def add(a b : Sign): Sign :=
  if a = .None v b = .None then
    .None
  else
    {
      hasZero := a.hasZero && b.hasZero ||
                a.hasNeg  && b.hasPos  ||
                a.hasPos  && b.hasNeg
      -- Safe since we assume the other is non-empty
      hasPos  := a.hasPos  || b.hasPos
      -- Safe since we assume the other is non-empty
      hasNeg  := a.hasNeg  || b.hasNeg
    }
```

```
theorem add_correct
: Sign.gc.IsBinAbstraction (· + ·) Sign.add
:= by // ...
```

Sign domain - Implementation

```
def neg(a: Sign): Sign := {  
  a with  
  hasNeg := a.hasPos  
  hasPos := a.hasNeg  
}
```

```
def neg_complete  
: Sign.gc.IsBestAbstraction (-·) Sign.neg  
:= by  
  intros x  
  simp only [Neg.neg, Sign.concrete, Set.preimage_setOf_eq, Sign.neg]  
  grind [Sign.neg, Sign.concrete]
```

Sign domain - Implementation

```
def mul (a b: Sign): Sign := {  
  hasPos  := a.hasPos && b.hasPos ||  
            a.hasNeg && b.hasNeg  
  hasZero := a.hasZero || b.hasZero,  
  hasNeg  := a.hasNeg && b.hasPos ||  
            a.hasPos && b.hasNeg  
}
```

```
theorem mul_correct  
: Sign.gc.IsBinAbstraction (· * ·) Sign.mul  
:= by // ...
```

For more examples, see the github repository



 `arthur-adjedj/lustrean`

Mathlib - What else can we get out of Mathlib?

```
class Lattice
  (α : Type u) extends SemilatticeSup α, SemilatticeInf α :
    Type u
```

[source](#)

A lattice is a join-semilattice which is also a meet-semilattice.

```
le : α → α → Prop
lt : α → α → Prop
le_refl (a : α) : a ≤ a
le_trans (a b c : α) : a ≤ b → b ≤ c → a ≤ c
lt_iff_le_not_ge (a b : α) : a < b ↔ a ≤ b ∧ ¬b ≤ a
le_antisymm (a b : α) : a ≤ b → b ≤ a → a = b
sup : α → α → α
le_sup_left (a b : α) : a ≤ sup a b
le_sup_right (a b : α) : b ≤ sup a b
sup_le (a b c : α) : a ≤ c → b ≤ c → sup a b ≤ c
inf : α → α → α
inf_le_left (a b : α) : Lattice.inf a b ≤ a
inf_le_right (a b : α) : Lattice.inf a b ≤ b
le_inf (a b c : α) : a ≤ b → a ≤ c → a ≤ Lattice.inf b c
```

► Instances

Seems to have all we need, we're just lacking laws about $\top$ and $\perp$

Mathlib - What else can we get out of Mathlib?

```
class BoundedOrder source  
  (α : Type u) [LE α] extends OrderTop α, OrderBot α :  
  Type u
```

A bounded order describes an order ($\leq$) with a top and bottom element, denoted $\top$ and $\bot$ respectively.

```
top : α  
le_top (a : α) : a ≤ top  
bot : α  
bot_le (a : α) : bot ≤ a
```

► Instances

We can combine this with `Lattice` to obtain something close to our `BoundedLattice` definition!

Mathlib - What else can we get out of Mathlib?

```
def BoundedLattice.ofLatticeAndBoundedOrder {α: Type}[Lattice α][BoundedOrder α]
: BoundedLattice α where
  // ...
  join_commutative := by grind
  join_associative := by grind
  join_absorption x y := by grind only [inf_le_left, sup_of_le_left]
  join_bot := by grind only [bot_le, sup_of_le_left]
  join_top := by simp
  meet_commutative := by grind
  meet_associative := by grind
  meet_absorption x y := by simp
  meet_bot := by simp
  meet_top := by simp
  join_is_lub := by grind [left_eq_inf, sup_le_iff]
  meet_is_glb := by grind [left_eq_inf, le_inf_iff]
```


Demo

```
def concrete(p: Partition D0 D1): Set C :=  
  { e | ∀ d : D0, e ∈ γ0 d → e ∈ γ1 (p d) }
```

```
def abstract(X: Set C): Partition D0 D1 :=  
  λ x ↦ α1 (X ∩ γ0 x)
```

Partition domain

```
def gcOfSubterms(gc₀: GaloisConnection  $\alpha_0$   $\gamma_0$ )(gc₁: GaloisConnection  $\alpha_1$   $\gamma_1$ )
: GaloisConnection ( $\beta := \text{Partition } D_0 \ D_1$ ) ( $\alpha := \text{Set } C$ )
  (abstract ( $\alpha_1 := \alpha_1$ ) ( $\gamma_0 := \gamma_0$ ))
  (concrete ( $\gamma_0 := \gamma_0$ ) ( $\gamma_1 := \gamma_1$ )) := by
  have  $\alpha_0\_mon := gc_0.monotone\_l$ 
  have  $\alpha_1\_mon := gc_1.monotone\_l$ 
  have  $\gamma_0\_mon := gc_0.monotone\_u$ 
  have  $\gamma_1\_mon := gc_1.monotone\_u$ 
  intros X Y
  dsimp [LE.le, abstract, concrete, Set.Subset, Subset]
  constructor <;> intros hyp
  · intros e e_X z e_Cz
    specialize hyp z
    rw [gc₁] at hyp; apply hyp; simp [*]
  · intros z; rw [gc₁]; intros e e_int
    simp only [Set.mem_inter_iff] at e_int
    apply hyp <;> simp [*]
```

Future work

-



 [arthur-adjedj/lustrean](https://github.com/arthur-adjedj/lustrean)